

RCE on Neon

Platform:

Neon is a **cloud-native, serverless PostgreSQL platform** designed for modern applications that require scalable, highly available, and easy-to-manage relational databases. It provides the core functionality of PostgreSQL while extending it with features tailored for cloud environments.

Note: The instance used for attack is a free-tier Neon database hosted on Azure.

(<https://neon.com/>)

Class:

Remote Code Execution

Summary:

We discovered a privilege-escalation vulnerability in the Neon-supported extension **anon**, which allowed us to obtain **superuser privileges** and subsequently achieve **command execution** inside **neonvm** by leveraging the **file_fdw** extension. Although the environment is similar to KubeVirt which means VM escape is difficult, based on the fact that all APIs currently have authentication enabled, improper network isolation still results in significant security risks like below.

Risk1: Due to the lack of restrictions on access to Azure' s metadata service at **169.254.169.254**, this created a potential **account takeover risk**.

Risk2: The PTR records exposed by Kubernetes' internal DNS leaked information about almost all active services.

Risk3: Grafana Alloy is unauthenticated and exposes network access to any pod.

Note: Due to time and resource constraints, there may be undiscovered risks.

Attack Description:

Phase I : Get RCE on Neonvm

First, create a neon database instance hosted on azure, then connect to it. Set anon extension enabled, and using PostgreSQL' s function overloading mechanism, we created a

function with the same name under the **anon** schema. The call function load_fake_data which feature is "security definer" and owner is superuser "cloud_admin". After that, Because of PostgreSQL's function argument matching feature, the user-created function is executed with the privileges of the superuser `cloud_admin` , thereby gaining superuser access showed in **Figure 1.**

pg_database_owner	[]	[v]	[]	[]	[]
pg_read_all_data	[]	[v]	[]	[]	[]
pg_write_all_data	[]	[v]	[]	[]	[]
pg_monitor	[]	[v]	[]	[]	[]
pg_read_all_settings	[]	[v]	[]	[]	[]
pg_read_all_stats	[]	[v]	[]	[]	[]
pg_stat_scan_tables	[]	[v]	[]	[]	[]
pg_read_server_files	[]	[v]	[]	[]	[]
pg_write_server_files	[]	[v]	[]	[]	[]
pg_execute_server_p	[]	[v]	[]	[]	[]
pg_signal_backend	[]	[v]	[]	[]	[]
pg_checkpoint	[]	[v]	[]	[]	[]
pg_maintain	[]	[v]	[]	[]	[]
pg_use_reserved_co	[]	[v]	[]	[]	[]
pg_create_subscripti	[]	[v]	[]	[]	[]
cloud_admin	[v]	[v]	[v]	[v]	[v]
neon_service	[]	[v]	[v]	[v]	[v]
neon_superuser	[]	[v]	[v]	[v]	[]
neondb_owner	[v]	[v]	[v]	[v]	[v]
authenticator	[]	[]	[]	[]	[v]
anonymous	[]	[v]	[]	[]	[]
authenticated	[]	[v]	[]	[]	[]

Figure1: make user default role neondb_owner with superuser

By leveraging the features of extension `file_fdw` , command execution can be achieved like in **Figure 2.**

USER=root
LC_TIME=C
AUDIT_LOGGING_ENDPOINT=audit-syslog-collector.neon-audit-logs.svc:10514
SHLVL=2
HOME=/var/db/postgres
LC_CTYPE=C.UTF-8
LC_MONETARY=C
NEON_AUTH_TOKEN=eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9.eyJzY29wZSI6InRlbnFudCIsInRlbnFudF9pZCI6IjFiNjU3Y2Q1Y2NjN
NEON_CONTROL_PLANE_TOKEN=eyJhbGciOiJFZERTQSIsInR5cCI6IkpXVCJ9.eyJlbnRwb2ludF9pZCI6IiIsImNvbXB1dGVfaWQiOiJjb2
RUST_LOG=info
TERM=vt102
PGSYSCONFDIR=/usr/local/etc
LC_COLLATE=C.UTF-8
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
OTEL_SDK_DISABLED=true
LANG=en_US.utf8
LC_MESSAGES=C.UTF-8
SHELL=/bin/sh
AUDIT_LOGGING_TLS_ENDPOINT=audit-syslog.aks0.westus3.internal.azure.neon.tech:10515
AUTOSCALING=true
PWD=/var/db/postgres/compute/pgdata
LC_NUMERIC=C

Figure2: execute command "env"

Phase II : Probing for Reachable Services

Given that we had already achieved command execution, and based on publicly available knowledge of the open-source stack, we understood that the environment was a KubeVirt-like virtual machine, making VM escape highly difficult. Therefore, we shifted our focus to the network attack surface. Our first step was to use PTR-based techniques to enumerate and identify active services running within the Kubernetes cluster. And some example information is shown in **Figure 3**.

```
10.22.12.170 [('10-22-12-170.compute-sparkling-night-aan5m2u.default.svc.cluster.local.',)]
10.22.13.175 [('10-22-13-175.neon-proxy-auth-broker-exposed.neon-proxy.svc.cluster.local.',), ('10-22-13-175.neon-proxy-auth-broker-exposed.neon-proxy.svc.cluster.local.',)]
10.22.14.0 [('10-22-14-0.neon-proxy-rest-broker-exposed.neon-proxy.svc.cluster.local.',), ('10-22-14-0.neon-proxy-rest-broker-exposed.neon-proxy.svc.cluster.local.',)]
10.22.14.12 [('10-22-14-12.neonvm-webhook-service.neonvm-system.svc.cluster.local.',)]
10.22.14.94 [('10-22-14-94.vmagent-prometheus-node-exporter.monitoring.svc.cluster.local.',)]
10.22.14.95 [('10-22-14-95.compute-raspy-breeze-aak1x6h7.default.svc.cluster.local.',)]
10.22.14.97 [('10-22-14-97.compute-snowy-truth-aa5z96zm.default.svc.cluster.local.',)]
10.22.14.123 [('10-22-14-123.nginx-ext-controller-metrics.nginx-ext.svc.cluster.local.',), ('10-22-14-123.nginx-ext-controller-metrics.nginx-ext.svc.cluster.local.',)]
10.22.14.150 [('neon-control-plane-api-0.neon-control-plane-api-headless.neon-control-plane.svc.cluster.local.',)]
10.22.64.7 [('pageserver-3.internal.cloudapp.net.',)]
10.22.64.8 [('pageserver-7.internal.cloudapp.net.',)]
10.22.64.9 [('pageserver-5.internal.cloudapp.net.',)]
10.22.64.10 [('pageserver-8.internal.cloudapp.net.',)]
10.22.64.11 [('pageserver-6.internal.cloudapp.net.',)]
10.22.64.12 [('pageserver-4.internal.cloudapp.net.',)]
10.22.66.4 [('safekeeper-1.internal.cloudapp.net.',)]
10.22.66.5 [('safekeeper-0.internal.cloudapp.net.',)]
10.22.66.6 [('safekeeper-2.internal.cloudapp.net.',)]
10.22.68.4 [('vpngw000000.internal.cloudapp.net.',)]
10.22.68.5 [('vpngw000001.internal.cloudapp.net.',)]
10.22.69.5 [('vm000004.internal.cloudapp.net.',)]
10.22.69.6 [('vm000005.internal.cloudapp.net.',)]
```

Figure3: PTR information in K8S

Next, we used this information to perform open-port probing and found that almost only the **metrics-related ports** were accessible; peer tenant service interfaces were all unreachable. From the metrics information shown in **Figure 4**, we identified the presence of **Grafana Alloy**, which had unauthenticated access exposed on ports **12345**, **4317**, and **4318**. It should be noted that some exposed services had unclear purposes. For ethical reasons, **we did not perform any brute-force probing against them**.

```
ce="default",service="compute-summer-meadow-aasgypho",uid="4d2d21c7-9d19-4b1c-bcd0-d02b14ca10da",cluster_ip="10.0.13.1",
ce="default",service="compute-delicate-dawn-aao9zwpr",uid="927817a5-983a-4c3a-a635-7c60d14976e4",cluster_ip="10.0.25.1",
ce="neon-telemetry",service="neon-telemetryapi",uid="c1282891-6aa8-467c-9245-63453e51c201",cluster_ip="10.0.73.171",
ce="grafana-alloy",service="grafana-alloy",uid="36b0f36d-4ccd-43d6-b468-b2ef37a85904",cluster_ip="10.0.79.30",external_ip="10.0.79.30",
ce="default",service="compute-curly-brook-aaswihq9",uid="533d7308-e553-473d-996f-d6a83769ec32",cluster_ip="10.0.37.1",
ce="default",service="compute-broad-cake-aa2zrmqs",uid="1ee6a44d-e51f-48f4-8707-de568bd1425d",cluster_ip="10.0.235.1",
```

Figure4: Metric information from vmagent-kube-state

Phase III: Attacking Cloud Resources

In the Azure public cloud, a classic attack vector exists in the metadata service (metaserver). What surprised us was that the **neonvm** instances were able to access this service. Some of the metadata information we retrieved is shown in **Figure 5**.

```

"subscriptionId": "10000000-0000-0000-0000-000000000000",
"systemFaultDomain": "",
"tags": "aks-managed-consolidated-additional-properties:10000000-0000-0000-0000-000000000000;aks-managed-cre
"tagsList": [
  {
    "name": "aks-managed-consolidated-additional-properties",
    "value": "10000000-0000-0000-0000-000000000000"
  },
  {
    "name": "aks-managed-createOperationID",
    "value": "10000000-0000-0000-0000-000000000000"
  },
  {
    "name": "aks-managed-creationSource",
    "value": "vmssclient-aks-compute-v3-26496424-vmss"
  },
  {
    "name": "aks-managed-enable-imds-restriction",
    "value": "false"
  },
  {
    "name": "aks-managed-kubeletIdentityClientID",
    "value": "10000000-0000-0000-0000-000000000000"
  },
  {
    "name": "aks-managed-orchestrator",
    "value": "Kubernetes:1.31.1"
  }
]

```

Figure5: metadata information

By issuing an auth token from the metadata service, we found that its permissions — as shown in **Figure 6** — include **read access to all resources**.


```

"properties": {
  "roleName": "archerarolec8350122",
  "type": "CustomRole",
  "description": "Archera Custom Role v2",
  "assignableScopes": [
    "/providers/Microsoft.Management/managementGroups/c8350122-1697-4543-929a-d4a75d1bb552",
    "/subscriptions/9a2ac9b9-6e3e-4128-a3eb-9c0471bb145c"
  ],
  "permissions": [
    {
      "actions": [
        "*/read",
        "Microsoft.Consumption/*",
        "Microsoft.CostManagement/*",
        "Microsoft.Support/*",
        "Microsoft.Billing/billingPeriods/read",
        "Microsoft.Billing/billingProperty/read",
        "Microsoft.Resources/subscriptions/read",
        "Microsoft.Resources/subscriptions/resourceGroups/read",
        "Microsoft.Advisor/configurations/read",
        "Microsoft.Advisor/recommendations/read",
        "Microsoft.Management/managementGroups/read"
      ],
      "notActions": []
    }
  ],
  "createdOn": "2024-06-28T10:38:22.5174474Z",
  "updatedOn": "2024-10-15T09:49:27.1023680Z",
  "createdBy": "a4055110-5827-46ae-bc87-676c40341f0d",
  "updatedBy": "475d9562-c765-41f4-8123-fe15c0b57332"
},

```

Figure6: The auth token's permissions

With **read access to all resources**, sensitive cloud-stored credentials — such as **KMS keys**, **container registry information**, and other secrets — become exposed from an attacker's perspective.

We did **not proceed any further**. We believe that stopping at this point was appropriate, as going deeper could have caused **unnecessary disruption** to Neon's service operations.

Usage:

We include the SQL that achieves RCE, named **poc.sql**, in the appendix.

Thinking:

In multi-tenant database scenarios on the cloud, cross-tenant attacks and defenses are typically multi-layered. In this case, although Neon implemented strong protections at the container layer, the network-level leakage ultimately led to a chain of security risks.

Acknowledgement:

This vulnerability was discovered by the **Volcano Engine Cloud Security Team**.

代码块

```
1  RCE on Neon
2
3  ## Platform:
4
5  **Neon** is a **cloud-native, serverless PostgreSQL platform** designed for
modern applications that require scalable, highly available, and easy-to-
manage relational databases. It provides the core functionality of PostgreSQL
while extending it with features tailored for cloud environments.
6  Note: The instance used for attack is a free-tier Neon database hosted on
Azure. ([https://neon.com/](https://neon.com/))
7
8  ## Class:
9
10 Remote Code Execution
11
12 ## Summary:
13
14 We discovered a privilege-escalation vulnerability in the Neon-supported
extension anon, which allowed us to obtain superuser privileges and
subsequently achieve command execution inside neonvm by leveraging the
file_fdw extension. Although the environment is similar to KubeVirt which
means VM escape is difficult, based on the fact that all APIs currently have
authentication enabled, improper network isolation still results in
significant security risks like below.
15
16 - Risk1: Due to the lack of restrictions on access to Azure's metadata service
at 169.254.169.254, this created a potential account takeover risk.
17 - Risk2: The PTR records exposed by Kubernetes' internal DNS leaked
information about almost all active services.
18 - Risk3: Grafana Alloy is unauthenticated and exposes network access to any
pod.
19 **Note:** Due to time and resource constraints, there may be undiscovered
risks.
20
21 ## Attack Description:
22
23 ### Phase I : Get RCE on Neonvm
24
25 First, create a neon database instance hosted on azure, then connect to it.
Set anon extension enabled, and using PostgreSQL's function overloading
mechanism, we created a function with the same name under the **anon** schema.
The call function load_fake_data which feature is \"security definer\" and
owner is superuser \"cloud_admin\". After that, Because of PostgreSQL's
```

function argument matching feature, the user-created function is executed with the privileges of the superuser cloud_admin, thereby gaining superuser access showed in **Figure 1**.

image.png

Figure1: make user default role neondb_owner with superuser

By leveraging the features of extension file_fdw, command execution can be achieved like in **Figure 2**.

image.png

Figure2: execute command "env"

Phase II: Probing for Reachable Services

Given that we had already achieved command execution, and based on publicly available knowledge of the open-source stack, we understood that the environment was a KubeVirt-like virtual machine, making VM escape highly difficult. Therefore, we shifted our focus to the network attack surface. Our first step was to use PTR-based techniques to enumerate and identify active services running within the Kubernetes cluster. And some example information is shown in **Figure 3**.

image.png

Figure3: PTR information in K8S

Next, we used this information to perform open-port probing and found that almost only the metrics-related ports were accessible; peer tenant service interfaces were all unreachable. From the metrics information shown in Figure 4, we identified the presence of Grafana Alloy, which had unauthenticated access exposed on ports 12345, 4317, and 4318. It should be noted that some exposed services had unclear purposes. For ethical reasons, we did not perform any brute-force probing against them.

image.png

Figure4: Metric information from vmagent-kube-state

Phase III: Attacking Cloud Resources

In the Azure public cloud, a classic attack vector exists in the metadata service (metaserver). What surprised us was that the neonvm instances were able to access this service. Some of the metadata information we retrieved is shown in **Figure 5**.

image.png

Figure5: metadata information

By issuing an auth token from the metadata service, we found that its permissions – as shown in Figure6 – include read access to all resources.

image.png

Figure6: The auth token's permissions

With read access to all resources, sensitive cloud-stored credentials – such as KMS keys, container registry information, and other secrets – become exposed from an attacker's perspective.

We did not proceed any further. We believe that stopping at this point was appropriate, as going deeper could have caused unnecessary disruption to Neon's service operations.

Usage:

We include the SQL that achieves RCE, named poc.sql, in the appendix.

Thinking:

In multi-tenant database scenarios on the cloud, cross-tenant attacks and defenses are typically multi-layered. In this case, although Neon implemented strong protections at the container layer, the network-level leakage ultimately led to a chain of security risks.

Acknowledgement:

This vulnerability was discovered by the **Volcano Engine Cloud Security Team**.